

The First Live Build

The moment the runner came online, it started working through every push queued since this morning — and surfaced two real bugs the pipeline had never actually hit before.

run: build #7 — commit 60bac55 2026-08-16

00 What was actually happening

Every commit pushed to `main` today had already queued its own `build` job — there just wasn't a runner to claim any of them until it was finally installed and started (doc 8). The instant it went active, it began draining that backlog, one job per push, in order. The failure emails weren't a new problem appearing — they were the pipeline's genuine first execution, discovering things local testing never could, because local testing always had a real `.env` sitting in the working directory and a Docker daemon already configured to trust the registry.

01 Bug 1 — a fresh checkout has no `.env`

Every `${VAR}` in `docker-compose.yml` needs a source. Locally, that's always been a real `.env` file sitting right there. A CI checkout is different — `.env` is gitignored on purpose (docs 2 and 7 both cover why), so it simply doesn't exist in the fresh clone the runner just checked out.

DEV LAPTOP — real `.env` present

XLSX_HOST_PATH=./coder_commands.xlsx
mount spec resolves correctly

FRESH CI CHECKOUT — no `.env` at all

XLSX_HOST_PATH=
mount spec becomes `"/data/...:ro"`

most blank vars → Compose accepts an empty

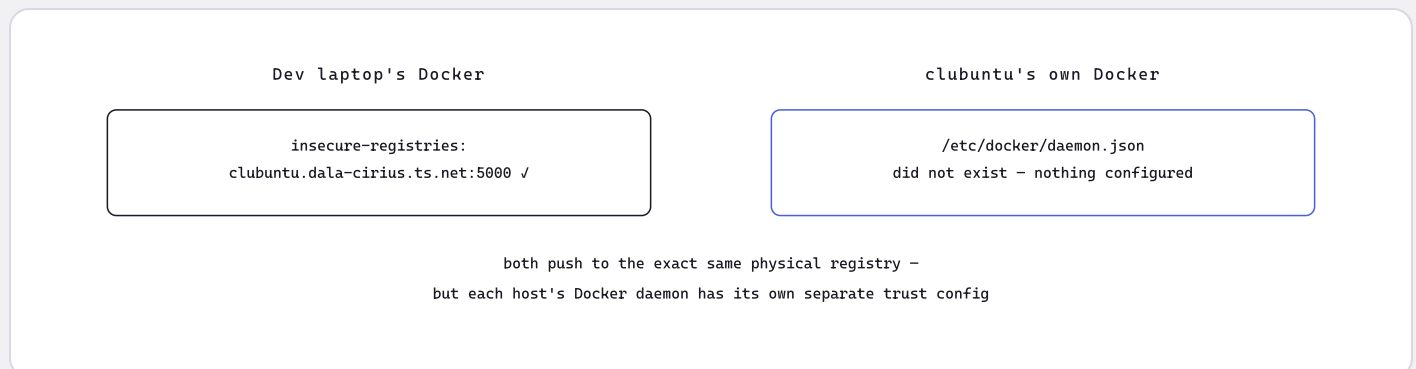
value — just a warning `XLSX_HOST_PATH` blank → empty *field* in a colon-delimited spec — a parse error invalid spec: `:/data/coder_commands.xlsx:ro:` empty section between colons

Fixed in `build.yml`: export throwaway placeholder values before `docker compose build` runs. `build` never starts a container, so these values are never used for anything real — they only need to be non-empty so the file parses.

02 Bug 2 — the push itself, HTTPS vs. plain HTTP

With the parse error fixed, the build genuinely ran — all five images built successfully. The failure moved to the very last step: `docker push`, with `http: server gave HTTP response to HTTPS client`. Docker assumes any registry speaks HTTPS unless explicitly told otherwise; this registry has only ever spoken plain HTTP (§ established when `.env` was written — deliberately, relying on Tailscale for privacy instead of TLS).

This never surfaced from the dev laptop, because the dev laptop's own Docker was already configured to trust this registry — every manual `docker push` all along had that config quietly doing its job. clubuntu's *own* Docker daemon had never been told the same thing.



The runner runs *on* clubuntu, so `docker compose push` always uses clubuntu's own daemon — never the dev laptop's, no matter how well-configured that one already was.

03 Why a restart was actually necessary

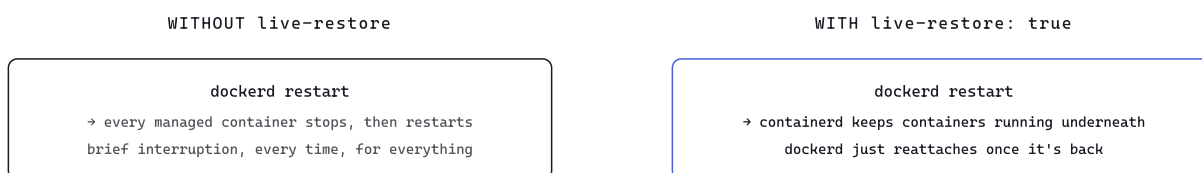
`daemon.json` is read once, at the moment `dockerd` starts — it isn't a live-reloadable file for a setting like this. Editing it on disk changes nothing about the daemon already running in memory; only a restart makes it read the file again and pick up the new trust list.



Not a peculiarity of this fix — true of most daemon-level Docker settings.

04 The real cost of that restart — and live-restore

By default, containers are tied to `dockerd`'s own lifecycle — restarting the daemon restarts everything it manages, and clubuntu runs a dozen other projects' containers, not just this one. `"live-restore": true` changes that relationship: containers actually run under `containerd`, a lower-level daemon `dockerd` normally supervises; with live-restore on, `containerd` keeps them running independently even while `dockerd` itself restarts.



This one restart still briefly interrupted everything (the old daemon hadn't loaded live-restore yet) — but every *future* restart on this host, for any reason, won't touch running containers again.

Verified after: all containers back within ~32 seconds, none lost, `docker info` confirms both `clubuntu.dala-cirius.ts.net:5000` and `live-restore` active.

05 Verified

CHECK	RESULT
Manual push after the daemon fix	✓ succeeded — 12 layers, real digest, exit 0
Registry catalog	✓ <code>postgres-mcp</code> now present

CHECK

RESULT

Other containers on
clubuntu

✓ **all survived the restart**

Full CI run, both bugs fixed
together

✓ **commit 811af77 — all five images built and pushed
automatically, zero manual steps**

```
$ curl registry catalog + tags, after the natural push to  
main for this doc's own commit:  
sync, coder-commands-mcp, docker-mcp, gateway → 811af77  
postgres-mcp → 811af77, 60bac55 (both tags coexist — the  
accumulation model working exactly as designed)
```

This is the build half of the pipeline fully proven: a plain `git push`, no human touching clubuntu at any point, five images built and in the registry. Deploy — the tag-triggered half — is next.

git-docs - 9 · run: build #7 (60bac55) → confirmed via 811af77